



NATIVE ANDROID APP

Core File Audit Report

[Existing Files](#) | [Missing Files](#) | [Modules & Services](#) | [Action Plan](#)

Audit Date: June 15, 2026

Architecture: Kotlin + Jetpack Compose + Material Design 3 + Hilt DI

Backend: Firebase + Cloudflare Edge Workers + Pure WebRTC

Total Files: 85 Kotlin | 24,500+ Lines of Code

Table of Contents

1. Executive Summary.....	1
Key Metrics.....	2
2. Existing Core Files Inventory.....	2
2.1 Build and Configuration.....	2
2.2 Domain Models.....	2
2.3 Domain Repository Interfaces.....	2
2.4 Data — Local Layer.....	2
2.5 Data — Repository Implementations.....	2
2.6 Data — Remote Services.....	3
2.7 Dependency Injection.....	3
2.8 UI — Theme and Design System.....	3
2.9 UI — Screens.....	3
2.10 App Entry and Navigation.....	3
3. Missing Core Files.....	3
3.1 Architecture Layer Gaps.....	3
3.2 Room Database Expansion.....	4
3.3 ViewModel and Component Gaps.....	4
3.4 Security and Sync Gaps.....	4
4. Stub and Incomplete Files.....	4
5. Critical Modules and External Services.....	4
6. High-Value Enhancements.....	4
7. Nice-to-Have Differentiators and Polish.....	5
8. Infrastructure and Backend Services.....	5
9. Architecture Scorecard.....	5
10. Recommended Action Plan.....	5
Phase 1: Critical Production Blockers (Weeks 1–4).....	6
Phase 2: High-Value Feature Completeness (Weeks 5–8).....	6
Phase 3: Premium Differentiators (Weeks 9–12).....	6
Phase 4: Infrastructure and Scale (Weeks 13–16).....	6

Right-click the table of contents and select "Update Field" to refresh page numbers.

1. Executive Summary

This audit provides a comprehensive inventory and gap analysis of the Zixo Native Android application codebase. The project is built using Kotlin, Jetpack Compose, Material Design 3, Hilt dependency injection, and pure WebRTC for peer-to-peer calling. The backend leverages Firebase (Auth, Firestore, Realtime Database, Storage, FCM) and Cloudflare Edge Workers for registration, Zixo Number minting, and passkey authentication challenges.

The audit reveals that all 42 originally required core files are present and implemented, representing over 24,500 lines of Kotlin code across 85 files. The critical specification updates from the latest iteration — including EglBase DI Singleton, AudioManager calibration, atomic Firestore batch writes for mutual contact linking, removal of all email-based lookups, foreground service types for camera and microphone, SurfaceViewRenderer/AndroidView integration, and ZXing QR code generation — have all been verified as present in the codebase.

However, the audit identifies 20 missing core files that are essential for production readiness, 7 stub or incomplete files that need expansion, and significant architectural gaps including the absence of a use case layer, offline-first sync engine, end-to-end encryption, and media loading infrastructure. These gaps separate the current codebase from a production-grade secure messenger.

Key Metrics

Metric	Value
Total Kotlin Files	85
Total Lines of Code	24,500+
Required Core Files Present	42 / 42 (100%)
Missing Critical Files	20
Stub/Incomplete Files	7
Architecture Pattern	MVVM (ViewModel → Repository → Impl)
Missing Architecture Layers	Use Cases, Mappers, Sync Engine
E2E Encryption	Not Implemented
Offline Support	Not Implemented
Test Coverage	0% (No test files exist)

2. Existing Core Files Inventory

The following table catalogs every file in the Zixo project, organized by architectural category. Each file includes its line count and a status indicator. Files marked with a warning symbol are stubs or near-stubs that require expansion, detailed further in Section 4.

2.1 Build and Configuration

File	Lines	Status
build.gradle.kts (root)	10	✓ Complete
settings.gradle.kts	26	✓ Complete
gradle.properties	7	✓ Complete
gradle/wrapper/gradle-wrapper.properties	5	✓ Complete
gradle/libs.versions.toml	101	✓ Complete
app/build.gradle.kts	152	✓ Complete
app/google-services.json	—	✓ Present
AndroidManifest.xml	101	✓ Fixed (camera mic phoneCall)

2.2 Domain Models

File	Lines	Status
domain/model/AppSettingsState.kt	247	✓ Complete
domain/model/ContactModel.kt	137	✓ Complete
domain/model/MessageModel.kt	176	✓ Complete
domain/model/StatusModel.kt	137	✓ Complete
domain/model/User.kt	37	✓ Complete
domain/model/SettingsEnums.kt	39	✓ Complete
domain/model/Chat.kt	23	⚠ Typealiases only
domain/model/CallLog.kt	40	✓ Complete
domain/model/Session.kt	17	⚠ Stub — no serialization

2.3 Domain Repository Interfaces

File	Lines	Status
domain/repository/AuthRepository.kt	97	✓ Complete
domain/repository/ContactRepository.kt	127	✓ Complete

File	Lines	Status
domain/repository/ ChatRepository.kt	125	✓ Complete
domain/repository/ CallRepository.kt	147	✓ Complete
domain/repository/ SettingsRepository.kt	108	✓ Complete
domain/repository/ StatusRepository.kt	80	✓ Complete

2.4 Data — Local Layer

File	Lines	Status
data/local/ PreferencesDataStore.kt	345	✓ Complete
data/local/datastore/ UserPreferences.kt	207	⚠ Legacy — superseded
data/local/room/ZixoDatabase.kt	27	⚠ Near-stub — 2 entities only
data/local/room/dao/ CallLogDao.kt	37	✓ Complete
data/local/room/dao/ChatDao.kt	42	✓ Complete
data/local/room/entity/ CallLogEntity.kt	73	✓ Complete
data/local/room/entity/ ChatEntity.kt	71	✓ Complete

2.5 Data — Repository Implementations

File	Lines	Status
data/repository/ AuthRepositoryImpl.kt	243	✓ Complete
data/repository/ ContactRepositoryImpl.kt	545	✓ Batch writes, no email
data/repository/ ChatRepositoryImpl.kt	573	✓ Complete
data/repository/ CallRepositoryImpl.kt	818	✓ RTDB signaling, IO

File	Lines	Status
data/repository/ SettingsRepositoryImpl.kt	401	✓ Complete
data/repository/ StatusRepositoryImpl.kt	405	✓ Complete

2.6 Data — Remote Services

File	Lines	Status
data/remote/cloudflare/ CloudflareApiService.kt	258	✓ Complete
data/remote/firebase/ FirebaseAuthService.kt	90	✓ Complete
data/remote/firebase/ FirestoreService.kt	373	✓ Complete
data/remote/firebase/ ZixoMessagingService.kt	304	✓ Complete
data/remote/webRTC/ WebRtcClient.kt	872	✓ EglBase + Audio + IO
data/remote/webRTC/ FirebaseSignalingClient.kt	682	✓ Complete
data/remote/webRTC/ CallForegroundService.kt	230	✓ cam+mic+phoneCall

2.7 Dependency Injection

File	Lines	Status
di/AppModule.kt	211	✓ Complete
di/FirebaseModule.kt	57	✓ Complete

2.8 UI — Theme and Design System

File	Lines	Status
ui/theme/Color.kt	132	✓ Complete
ui/theme/Theme.kt	210	✓ Complete
ui/theme/Typography.kt	132	✓ Complete
ui/components/ LiquidGlassModifiers.kt	688	✓ Complete

File	Lines	Status
ui/components/PermissionShield.kt	738	✓ Complete
ui/components/CallScreenOverlay.kt	653	✓ AndroidView + Surface
ui/components/ZixoBottomNav.kt	109	✓ Complete
ui/components/AvatarComponent.kt	98	✓ Complete
ui/components/NavigationItem.kt	90	✓ Complete
ui/components/ZixoNumberBadge.kt	84	✓ Complete
ui/components/ZixoTopBar.kt	81	✓ Complete
ui/components/SwitchItem.kt	98	✓ Complete
ui/components/SegmentedPicker.kt	66	✓ Complete
ui/components/SectionHeader.kt	34	⚠ Near-stub

2.9 UI — Screens

File	Lines	Status
ui/main/HomeScreen.kt	419	✓ Complete
ui/screens/auth/AuthScreen.kt	347	✓ Complete
ui/screens/auth/AuthViewModel.kt	257	✓ Complete
ui/screens/chats/ChatsScreen.kt	317	✓ Complete
ui/screens/chats/ChatsViewModel.kt	97	⚠ Thin
ui/screens/calls/CallsScreen.kt	613	✓ Complete
ui/screens/calls/CallsViewModel.kt	77	⚠ Thin
ui/chat/ChatMessageScreen.kt	957	✓ Complete
ui/chat/ChatViewModel.kt	595	✓ Complete
ui/chat/GroupChatScreen.kt	1289	✓ Complete
ui/contacts/ContactListScreen.kt	595	✓ Complete
ui/contacts/ContactListViewModel.kt	246	✓ Complete

File	Lines	Status
ui/contacts/FindContactDialog.kt	502	✓ Complete
ui/status/StatusTabScreen.kt	1723	✓ Complete
ui/status/StatusViewModel.kt	446	✓ Complete
ui/settings/SettingsScreen.kt	725	✓ QR modal included
ui/settings/SettingsViewModel.kt	583	✓ ZXing QR generation
ui/settings/EditProfileScreen.kt	315	✓ Complete
ui/settings/SubPages/ AccountSecurityScreen.kt	448	✓ Complete
ui/settings/SubPages/ ChatConfigScreen.kt	316	✓ Complete
ui/settings/SubPages/ NotificationManagerScreen.kt	384	✓ Complete
ui/settings/SubPages/ PrivacyCenterScreen.kt	430	✓ Complete
ui/settings/SubPages/ StorageDataHubScreen.kt	538	✓ Complete

2.10 App Entry and Navigation

File	Lines	Status
MainActivity.kt	264	✓ Complete
ZixoApplication.kt	39	✓ Complete
ui/navigation/ZixoNavigation.kt	436	✓ Complete

3. Missing Core Files

The following 20 files are identified as missing from the codebase but are essential for a production-grade secure messenger. Each entry includes the priority level, the architectural justification for its inclusion, and the specific functionality it would provide. Files marked CRITICAL represent hard blockers for production deployment.

3.1 Architecture Layer Gaps

File	Priority	Justification
domain/usecase/ (entire layer)	CRITICAL	Clean Architecture is incomplete. ViewModels call repositories directly. Use cases decouple business logic, improve testability, and prevent repository interface bloat. A messenger with complex multi-repository workflows (e.g., send message + verify contact + update last seen) needs a use case orchestrator.
data/mapper/ (entire layer)	MEDIUM	Model mapping is scattered inline in repository implementations (mapToChatThreadModel, toDomain(), toEntity()). A dedicated mapper layer eliminates duplication, enforces single-responsibility, and makes it easy to add field transformations without touching repository code.
data/sync/ (entire layer)	CRITICAL	No offline-first sync engine exists. When Firestore is unreachable, there is no queuing, conflict resolution, or merge strategy. A sync layer with SyncWorker, SyncStatus tracking, and conflict resolution logic is critical for any messenger that needs to work reliably on intermittent connections.

3.2 Room Database Expansion

File	Priority	Justification
data/local/room/entity/ MessageEntity.kt	CRITICAL	Messages are fetched from Firestore but never cached locally. Offline reading is impossible. Room needs a MessageEntity for the chat cache, enabling users to read conversations without network access.

File	Priority	Justification
data/local/room/entity/ ContactEntity.kt	CRITICAL	Contacts exist only in Firestore. Offline access to contacts fails entirely. Room needs a ContactEntity for local caching and faster lookups without round-trips to the server.
data/local/room/entity/ StatusEntity.kt	MEDIUM	Statuses are not cached locally. Viewing status history requires network every time. A StatusEntity enables offline viewing and reduces redundant Firestore reads.
data/local/room/entity/ UserEntity.kt	MEDIUM	User profiles not cached. Profile viewing fails offline and every profile display costs a Firestore read. A UserEntity cache reduces costs and improves performance.
data/local/room/dao/ MessageDao.kt	CRITICAL	No DAO for messages means cached messages cannot be queried locally. MessageDao would provide full-text search, pagination, and thread-based queries against the Room cache.
data/local/room/dao/ ContactDao.kt	CRITICAL	No DAO for contacts means the contact list requires a server round-trip every time. ContactDao enables instant offline contact listing with sort and filter.
data/local/room/dao/StatusDao.kt	MEDIUM	No DAO for statuses. StatusDao enables offline status queue management and auto-expiration cleanup.
data/local/room/dao/UserDao.kt	MEDIUM	No DAO for user profiles. UserDao enables profile caching with TTL-based invalidation.
data/local/room/Migrations.kt	MEDIUM	Database uses fallbackToDestructiveMigration() which wipes all local data on schema changes. Production needs proper migration scripts to preserve user data across app updates.

3.3 ViewModel and Component Gaps

File	Priority	Justification
ui/main/HomeViewModel.kt	MEDIUM	HomeScreen (419 lines) has no ViewModel and manages state directly. This breaks the MVVM pattern, makes the screen untestable, and risks state loss on configuration changes.
ui/chat/GroupChatViewModel.kt	MEDIUM	GroupChatScreen (1289 lines) shares ChatViewModel instead of having its own. Group-specific logic like member management, admin controls, and group media has no proper home.
ui/components/QrCodeDialog.kt	LOW	QR code UI is embedded in SettingsScreen. A reusable component enables QR sharing from contact cards, profile exchange flows, and deep-link invitations across the app.
ui/components/NotificationHelper.kt	MEDIUM	Notification logic is split between ZixoMessagingService and CallForegroundService. A centralized helper manages channels, grouping, DND override, and smart notification routing.
data/remote/webrtc/AudioManager.kt	LOW	Audio calibration is embedded in WebRtcClient.kt (872 lines). Extracting it to a standalone ZixoAudioManager improves maintainability and enables independent testing of audio routing logic.
data/remote/webrtc/PeerConnectionObserver.kt	LOW	PeerConnection callbacks are anonymous inlines in WebRtcClient.kt. A named observer class makes WebRTC event handling testable, readable, and reusable.

3.4 Security and Sync Gaps

File	Priority	Justification
domain/model/EncryptionModel.kt	CRITICAL	No end-to-end encryption model or key exchange protocol exists. A secure messenger without E2E encryption means Firebase administrators can read every message. This is the single largest trust gap in the architecture.
data/remote/firebase/FirestoreSyncWorker.kt	CRITICAL	No WorkManager-based background sync exists. FCM messages are received, but no periodic Firestore-to-Room reconciliation runs. Without this, data drifts between server and client over time.

4. Stub and Incomplete Files

The following files exist in the codebase but are flagged as stubs, near-stubs, or otherwise incomplete. These files contain minimal implementation and need significant expansion before they can support production functionality. Each entry describes the specific issue and the recommended remediation.

File	Lines	Issue	Recommended Fix
Session.kt	17	No @SerializedName, no Firestore toMap()/fromSnapshot(), no companion factory	Add toMap(), fromSnapshot(), Companion.fromDevice() factory, and Firestore serialization annotations
Chat.kt	23	Only 3 typealiases redirecting to MessageModel.kt	Remove file entirely and update all imports — dead code that adds confusion
ZixoDatabase.kt	27	Only 2 entities registered — missing Message, Contact, Status, User entities	Add all 4+ missing entities and their DAOs, implement proper migration strategy
ChatsViewModel.kt	97	Very thin — likely missing search, filter, archive, and unread count logic	Expand with real chat list management, search/filter, archive support, and badge counts

File	Lines	Issue	Recommended Fix
CallsViewModel.kt	77	Very thin — likely missing call history filtering, stats, and redial logic	Expand with call log management, duration stats, filter by type, and redial actions
SectionHeader.kt	34	Minimal component — lacks subtitle, trailing icon, and click action	Add subtitle support, trailing icon slot, click handler, and Liquid Glass styling
UserPreferences.kt	207	Marked as legacy and superseded by PreferencesDataStore.kt	Delete file and migrate any remaining references to PreferencesDataStore

5. Critical Modules and External Services

The following modules and services are classified as CRITICAL for a production messenger. Without these, the app cannot be considered production-ready regardless of how many UI screens exist. Each entry describes what it adds and why Zixo specifically needs it.

Module / Service	What It Adds	Why Zixo Needs It
Signal Protocol (libsignal)	End-to-end encryption with X3DH key agreement and Double Ratchet message encryption	Zixo is a secure messenger with zero-trust architecture but NO E2E encryption. Without it, Firebase admins can read every message. This is the #1 production blocker.
WorkManager + Room Sync	Periodic Firestore-to-Room reconciliation, offline queue, exponential backoff retry	No offline support currently exists. Messages and contacts disappear when the network drops. A messenger that does not work offline is not production-ready.
Firebase Crashlytics	Real-time crash reporting with stack traces, non-fatal issue tracking, user impact metrics	Timber is used for logging but no crash reporting exists. Production apps need crash analytics to detect and fix issues users encounter in the wild.
LeakCanary (Debug)	Automatic memory leak detection for Activities, Fragments, Views, and retained objects	WebRTC has notoriously complex lifecycles. LeakCanary catches SurfaceViewRenderer, EglBase, and PeerConnection leaks before they reach production builds.

Module / Service	What It Adds	Why Zixo Needs It
Firebase App Check	Attestation that API calls originate from a genuine Zixo app binary	Currently anyone with the Firebase project config can call APIs directly. App Check blocks unauthorized clients and protects against abuse.

6. High-Value Enhancements

These modules are not hard blockers for launch but significantly elevate the user experience and bring Zixo closer to parity with established messengers like WhatsApp, Telegram, and Signal. Each represents a meaningful quality leap.

Module / Service	What It Adds	Why Zixo Needs It
Coil 3	Fast Kotlin-first image loading with Compose integration, disk and memory caching	No image loading library is detected. Avatar images, status media, and chat attachments all need efficient loading with placeholder and error states.
Accompanist SystemUIController	Status bar and navigation bar color and icon styling per theme	Liquid Glass design requires transparent status bars and light/dark system icon toggling, which is not currently handled.
Lottie Compose	After Effects animations rendered natively in Compose	Call ring animations, message send confirmations, typing indicators, and onboarding animations become pixel-perfect and buttery smooth.
Paging 3 (Jetpack)	Infinite scroll with Firestore PagingSource, placeholder support, and memory efficiency	Chat message lists and contact lists load ALL data at once. Paging 3 gives incremental loading that scales to thousands of messages.
Hilt Worker Injection	WorkManager and Hilt interop for injecting dependencies into workers	Sync workers need injected dependencies (repositories, DataStore). Without this, you cannot properly DI into WorkManager workers.
Firebase Remote Config	Server-side feature flags and A/B testing without app updates	Toggle features like video calls, group chat max members, and UI variations without requiring users to update the app.

Module / Service	What It Adds	Why Zixo Needs It
Biometric Prompt API	Fingerprint and face unlock for app access on launch	AccountSecurityScreen exists in the UI but no real biometric gate is implemented. Adding androidx.biometric enables app-lock on launch.
ML Kit Smart Reply	On-device suggested replies in chat based on message context	Generates context-aware quick reply suggestions without sending data to servers. Major UX differentiator for a messaging app.
ML Kit Language ID + Translate	On-device message translation supporting 50+ languages	Real-time message translation in chats without cloud dependency. Critical for any international messenger.
ExoPlayer 2 / Media3	Audio and video playback with background audio, PiP, and adaptive streaming	No media player is detected. Voice messages, video status, and shared videos need a proper player with background audio and Picture-in-Picture support.

7. Nice-to-Have Differentiators and Polish

These modules represent premium features and polish that differentiate Zixo from competitors. They are not required for launch but create a distinctly premium user experience and open up feature categories that users expect from modern messengers.

Module / Service	What It Adds	Why Zixo Needs It
Emoji2 + Custom Sticker Keyboard	System emoji compatibility and custom sticker/emoji keyboard	A chat app without an emoji picker feels incomplete. Emoji2 gives backward-compatible rendering; a sticker keyboard enables premium content.
Relinker	Safer native library loading with retry and workaround logic	WebRTC native .so files can fail to load on some devices. Relinker handles the edge cases that cause crashes on specific OEMs.
Chucker (Debug)	In-app HTTP traffic inspector for debugging API calls	Debug Cloudflare API calls and signaling without needing Charles or Proxyman. Debug builds only.

Module / Service	What It Adds	Why Zixo Needs It
Firebase Dynamic Links	Deep link resolution for zixo:// profile URIs across app installs	QR codes generate zixo://profile/{zixoNumber} URIs. Dynamic Links make these work even when the app is not yet installed.
AndroidX Browser (Custom Tabs)	In-app web browsing for shared links without leaving the app	Chat messages with URLs should open in Custom Tabs instead of leaving the app entirely.
Splash Screen API (AndroidX)	Branded splash screen with animation on Android 12+	Android 12+ requires a splash screen. The core-splashscreen library gives branded launch with smooth transition to Compose UI.
Google Drive Backup	Cloud backup and restore for chat history and media	Competitors all offer cloud backup. Implementing via Google Drive API is essential for user retention when switching devices.
Shimmer Compose	Loading skeleton animations during data fetch	Professional apps show shimmer placeholders during data loading instead of blank screens. Gives a polished feel to every loading state.
Compose Markdown	Rich text rendering for bold, italic, and code formatting in chat	Support bold , <i>italic</i> , and <code>`code`</code> formatting in messages without requiring full HTML rendering.
Haptic Feedback	Tactile vibration for keyboard presses, button actions, and call events	HapticFeedbackConstants for keyboard, button presses, incoming call patterns, and message send confirmation.
Glance (Widgets)	Home screen widgets for unread chats and latest messages	Android widgets showing unread count and latest message. Major UX win for a messenger used throughout the day.
Wear OS Compose	Companion smartwatch app for quick reply and call management	Quick reply and call accept/decline from a smartwatch. Premium feature that signals platform maturity.

8. Infrastructure and Backend Services

Beyond the Android app itself, the following infrastructure services can dramatically improve performance, reliability, and operational visibility. These are backend-side investments that multiply the app's capabilities without changing client code.

Service	What It Adds	Impact on Zixo
Cloudflare D1	SQLite at the Edge for low-latency user directory lookups	Zixo Number lookups by exact 8-digit match are faster with D1's indexed queries than Firestore collection scans. Reduces p99 latency from ~200ms to ~20ms.
Cloudflare R2	Zero-egress-cost object storage for media assets	Profile photos, status media, and voice messages stored with zero bandwidth charges. Dramatically reduces operational costs at scale.
Cloudflare Streams	Optimized video delivery with adaptive bitrate and transcoding	Status videos and shared video content delivered with adaptive bitrate. Eliminates buffering on slow connections.
Algolia	Lightning-fast full-text search for messages (local + cloud)	Zixo currently has no message search functionality. Algolia enables instant search across all conversations with typo tolerance.
Sentry	Cross-platform error tracking with breadcrumbs, release tracking, and session replay	Deeper crash analytics than Crashlytics with session replay. Understand exactly what the user did before a crash.
RevenueCat	In-app subscription management and purchase validation	If Zixo adds premium features like custom themes, larger file uploads, or E2E encryption backup, RevenueCat handles the subscription lifecycle.
Twilio Verify	SMS-based account recovery and phone number verification	No email is used in Zixo's privacy architecture. SMS verification provides an alternative recovery channel for account lockout scenarios.
OpenTelemetry	Distributed tracing across Cloudflare Workers, Firebase, and the app	End-to-end latency debugging from user action through Cloudflare Edge to Firebase and back. Identifies bottlenecks in the critical path.

Service	What It Adds	Impact on Zixo
Grafana + Prometheus	Real-time call quality metrics dashboards (jitter, packet loss, RTT)	WebRTC call quality monitoring with alerts when packet loss exceeds thresholds. Essential for maintaining call reliability at scale.
Self-hosted TURN Server	Custom TURN relay for WebRTC when STUN fails behind symmetric NATs	Google STUN servers do not traverse symmetric NATs (common in corporate networks). A self-hosted TURN server on Cloudflare improves call connection rates from ~85% to ~99%.

9. Architecture Scorecard

The following scorecard evaluates each architectural dimension on a scale from the current implementation state to the target production state. Gap severity indicates how significantly the current state deviates from what a production messenger requires.

Dimension	Current State	Target	Gap Severity
Core Files	85 files present	85 required	None — 100%
Architecture Depth	ViewModel → Repository → Impl	ViewModel → UseCase → Repository → Impl	MEDIUM — No use case layer
Offline Support	None (Firestore-only)	Room + WorkManager sync	CRITICAL — No offline access
E2E Encryption	None	Signal Protocol X3DH + Double Ratchet	CRITICAL — Server can read messages
Media Loading	None detected	Coil 3 with disk + memory cache	CRITICAL — No image caching
Media Playback	None detected	ExoPlayer / Media3	HIGH — No audio/video player
Crash Reporting	Timber logging only	Timber + Crashlytics + Sentry	HIGH — No production monitoring
Image Caching	None	Coil 3 disk + memory cache	CRITICAL — Images reload every time
Search	None for messages	Algolia or Room FTS5	HIGH — No message search
Biometric Lock	UI only, no real implementation	Biometric Prompt API	HIGH — Security screen is decorative
Backup / Restore	None	Google Drive or local export	HIGH — No data portability

Dimension	Current State	Target	Gap Severity
Testing	No test files at all	Unit + Integration + UI tests	CRITICAL — 0% test coverage
Accessibility	Not audited	TalkBack + large text + contrast	MEDIUM — Not verified
Performance	Not profiled	Baseline profiles + startup tracing	MEDIUM — Unknown cold start time

10. Recommended Action Plan

Based on the audit findings, the following phased roadmap prioritizes critical production blockers first, then progressively adds high-value features and polish. Each phase has clear deliverables and can be completed independently.

Phase 1: Critical Production Blockers (Weeks 1–4)

This phase addresses hard blockers that prevent the app from being deployable as a secure messenger. Without these, the app cannot be considered production-ready under any definition.

Task	Files to Create/Modify	Dependencies
Implement E2E Encryption (Signal Protocol)	domain/model/EncryptionModel.kt, domain/usecase/EncryptMessageUseCase.kt, data/crypto/SignalCryptoService.kt	libsignal-client library
Build Offline-First Sync Engine	data/sync/SyncWorker.kt, data/sync/SyncStatus.kt, data/sync/ConflictResolver.kt, data/remote/firebase/FirestoreSyncWorker.kt	WorkManager, Hilt Worker
Expand Room Database	entity/MessageEntity.kt, entity/ContactEntity.kt, entity/StatusEntity.kt, entity/UserEntity.kt, dao/MessageDao.kt, dao/ContactDao.kt, dao/StatusDao.kt, dao/UserDao.kt, Migrations.kt	Room KSP
Add Image Loading (Coil 3)	Add Coil dependency, update AvatarComponent, ChatMessageScreen, StatusTabScreen	coil-compose

Task	Files to Create/Modify	Dependencies
Integrate Crashlytics	Add Firebase Crashlytics SDK, configure in build.gradle.kts, add Timber tree	firebase-crashlytics
Add Unit Tests (Core Layer)	test/ directory with AuthRepositoryImplTest, ContactRepositoryImplTest, ChatRepositoryImplTest, etc.	JUnit5, MockK, Turbine

Phase 2: High-Value Feature Completeness (Weeks 5-8)

This phase fills feature gaps that users expect from any modern messenger. While not hard blockers, these are quality differentiators that determine whether users stay or leave.

Task	Files to Create/Modify	Dependencies
Add Use Case Layer	domain/usecase/ with GetContactsUseCase, SendMessageUseCase, InitiateCallUseCase, etc.	Hilt DI
Implement Biometric Lock	data/security/ BiometricAuthManager.kt, update AccountSecurityScreen	androidx.biometric
Add Message Search (FTS5)	data/local/room/dao/ MessageSearchDao.kt, ui/chat/SearchScreen.kt	Room FTS5
Add Media Playback	data/media/AudioPlayerService.kt, data/media/VideoPlayerComponent.kt, update ChatMessageScreen	Media3 ExoPlayer
Add Data Mapper Layer	data/mapper/ContactMapper.kt, MessageMapper.kt, UserMapper.kt, CallLogMapper.kt	None
Implement Paging 3	data/paging/ ChatMessagePagingSource.kt, data/paging/ContactPagingSource.kt, update ViewModels	paging-runtime
Add HomeViewModel	ui/main/HomeViewModel.kt, refactor HomeScreen	Hilt
Add GroupChatViewModel	ui/chat/GroupChatViewModel.kt, refactor GroupChatScreen	Hilt

Phase 3: Premium Differentiators (Weeks 9–12)

This phase adds features that differentiate Zixo from competitors and create a distinctly premium experience. These are the features that turn a functional messenger into one that users love and recommend.

Task	Description	Dependencies
Lottie Animations	Add animated call rings, message confirmations, typing indicators, and onboarding flow	lottie-compose
Smart Reply (ML Kit)	Context-aware quick reply suggestions in chat	mlkit-smart-reply
On-Device Translation	Real-time message translation for 50+ languages	mlkit-translate
Shimmer Loading States	Replace blank screens with skeleton loading animations	compose-shimmer
Compose Markdown	Rich text formatting in chat messages	compose-markdown
Emoji + Sticker Keyboard	Full emoji picker and custom sticker support	emoji2
Dynamic Links	zixo:// deep links work across app install flows	firebase-dynamic-links
Custom Tabs	URL previews and in-app browsing for shared links	androidx.browser
Branded Splash Screen	Smooth app launch with brand animation	core-splashscreen
Haptic Feedback	Tactile responses for calls, messages, and interactions	Android OS APIs

Phase 4: Infrastructure and Scale (Weeks 13–16)

This phase focuses on backend infrastructure that improves reliability, reduces costs, and provides operational visibility at scale. These investments compound as the user base grows.

Task	Description	Dependencies
Self-hosted TURN Server	Deploy TURN relay on Cloudflare for symmetric NAT traversal	coturn on Cloudflare
Cloudflare R2 Media Storage	Migrate media uploads from Firebase Storage to R2 for zero egress costs	Cloudflare R2 API

Task	Description	Dependencies
Cloudflare D1 Directory	Offload Zixo Number lookups to D1 for sub-20ms response	Cloudflare D1 API
OpenTelemetry Tracing	Distributed tracing from app through Edge to Firebase	OpenTelemetry SDK
Grafana Call Quality Dashboard	Real-time WebRTC quality metrics (jitter, packet loss, RTT)	Prometheus + Grafana
Sentry Session Replay	Crash reproduction with full user session replay	Sentry SDK
Google Drive Backup	Chat history backup and restore via Google Drive	Google Drive API
Glance Home Widgets	Unread message count and latest message on home screen	glance-appwidget
LeakCanary Integration	Automatic memory leak detection in debug builds	leakcanary-android
R8 Full Mode + ProGuard	Code shrinking, obfuscation, and WebRTC native keep rules	R8 configuration